SLLAB: A Platform for Running a College Club Sports Team

Lucius Scala

November 2026

College club sports teams are run by students, with no staff, little budget, and officers who change every year or two. SLLAB is a platform that gives such a club the logistics of a staffed program on a single backend. It consists of Fixture by SLLAB, an iOS app for schedules, availability, reminders and attendance; UCSDMCS by SLLAB, a public club website with live standings, a calendar feed, a web version of Fixture and an administration dashboard; and Film by SLLAB, an iOS app that records games from a phone mounted on a custom motorized base, steered by on-device object detection. The platform is in daily use by the UC San Diego Men's Club Soccer team, and Fixture is available on the App Store. This write-up describes the system's architecture and data model, the computer-vision and embedded control loop behind the camera, the engineering practices used to maintain it, and the plan for generalizing one team's tools into a platform that any club can adopt.

University of California, San Diego; lscala@ucsd.edu. *AI disclosure*: parts of this codebase and this write-up were written with the help of AI. All files were reviewed by a human, but they are provided as-is, with no guarantee that they are correct, secure, or fit for any particular purpose.

1. Introduction

The problem. A college club team does much of what a varsity program does: it schedules practices, books fields, travels to tournaments, recruits at tryouts and keeps its roster eligible. But students do all of it, in their spare time, with general-purpose tools: group chats for attendance, spreadsheets for rosters, a form for tryouts, and a website that is rarely updated. Each tool works on its own and nothing connects them, so the same information is entered in several places and drifts out of date. When officers graduate, much of what they knew leaves with them.

The cost of film. Game film is one of the most useful tools a team has for improving, and one of the least accessible. It requires either a person behind the camera for every match or a commercial auto-tracking camera, which costs around $2,000.

The approach. SLLAB is one connected platform with one backend. The schedule a captain creates is the schedule players respond to, the one the public website shows, and the one game film is attached to. The design goals are that a club can adopt it without a developer, that it works with the habits players already have, and that it is polished enough for the team to feel like a program.

What exists. Four components are built. Fixture by SLLAB is an iOS app for schedules, availability, reminders and attendance, released on the App Store at version 1.2. UCSDMCS by SLLAB is the club's public website, which also hosts a web version of Fixture and an administration dashboard. Film by SLLAB records games on a phone mounted on a motorized base and is a working prototype. Hardware by SLLAB is that base: ESP32 firmware, a custom printed circuit board and 3D-printed parts, costing about $400 in total.

Outline. Section 2 gives an overview of the system. Sections 3 and 4 describe Fixture and the club website. Section 5 describes the data model. Section 6 describes Film, its control loop and its hardware. Section 7 covers engineering practice, and Section 8 the roadmap. Appendix A gives a development timeline.

2. System Overview

Every component shares one Postgres database, hosted on Supabase (Supabase 2026). The iOS apps read and write the database directly, under access rules enforced by the database itself. Operations that require a secret, such as sending text messages, run on a

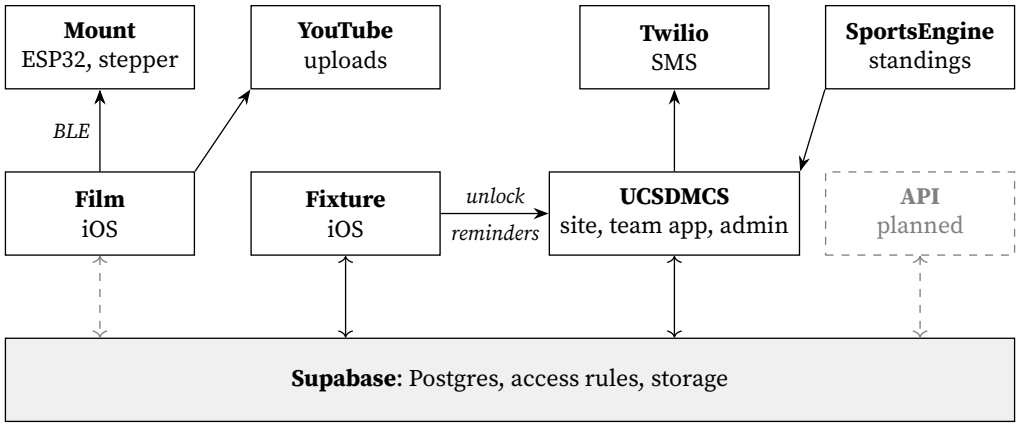


FIGURE 1. Architecture of the current system

Solid lines are in use; dashed lines and boxes are planned. Fixture and the website share the database. Fixture calls the website for the two operations that need a server: unlocking captain mode and sending reminders. Film does not yet use the shared backend.

TABLE 1. Components and their technology

Component	Technology	Role
Fixture by SLLAB	SwiftUI, supabase-swift	Player and captain app
UCSDMCS by SLLAB	Next.js 16, React 19, Tailwind CSS 4	Public site, web team app, administration, server endpoints
Film by SLLAB	SwiftUI, AVFoundation, Vision, Core ML, Core Bluetooth	Recording and tracking
Hardware by SLLAB	ESP32 (Arduino), TMC2209 driver, NEMA 17 stepper	Pan base and its controller
Detection model	Ultralytics YOLO26s, exported to Core ML	On-device detection
Database	Supabase, Postgres 17	Single source of truth

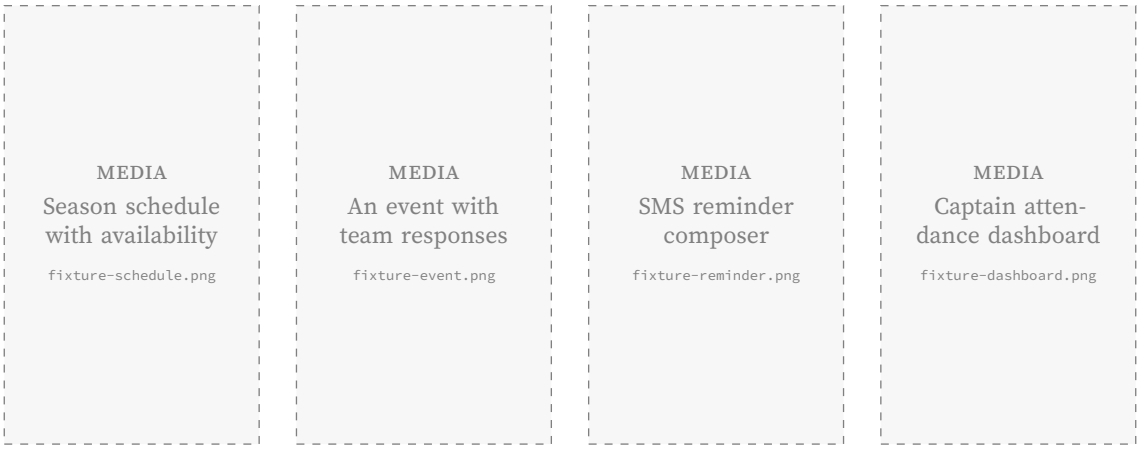
server instead. Figure 1 shows the system as it is today, and Table 1 lists each component's technology.

3. Fixture by SLLAB

Fixture is the app the team uses every day. Its central design constraint was adoption: a club cannot ask thirty students to create accounts before they can say whether they are coming to practice, so joining had to take seconds. Figure 2 shows its main screens.

3.1. Players

A player joins with a team code from their captain, picks and confirms their name on the roster, and adds a phone number. From then on, the app opens to the season's sched-



A. Schedule B. Event C. Reminder D. Attendance

FIGURE 2. Fixture by SLLAB on iOS

ule of practices and games, each with its field, a link to directions and the player's own response. Marking oneself going or not going takes one tap, and each event shows who else is coming.

3.2. Captains

Captain mode is unlocked with a separate code, which the server exchanges for a signed token. The app keeps the token in the iOS Keychain and sends it with requests the server performs on a captain's behalf, such as sending reminders. Captains can create games and practices, including a repeating practice schedule in which choosing weekdays and a date range creates every session at once. They can edit and delete events, see each event's responses grouped into going, not going and no response, send reminders, and track attendance over the season.

Reminders never expose phone numbers. The app sends the server a list of roster identifiers and a message, never phone numbers. The server looks the numbers up, sends the texts through a Twilio Messaging Service, which also handles opt-out replies (Twilio 2026), and returns how many were delivered. Numbers never reach the captain's device, and no one outside the roster can be texted through the endpoint.

Attendance is ranked by what a captain must act on. For a player with n events in the selected period, of which they declined d and left u unanswered, the dashboard orders players by

$$\text{issue score} = 1.5\,\frac{u}{n} + \frac{d}{n}. \tag{1}$$

An unanswered event weighs more than a declined one because a refusal lets a captain plan, while silence does not. Each player's page lists the events they attended and those they missed, with the reason.

3.3. Operating concerns

Universal links. Each reminder text contains a link of the form `app.ucsdmcs.com/events/<id>`, which opens the event directly in the app, or in the web version if the app is not installed.

Mandatory updates. The database stores a minimum supported version, and older builds show an update screen. The check fails open: if the app cannot reach the server, players are not locked out.

Tests. Unit tests cover the models, decoding of the database's JSON, availability grouping, attendance statistics and the data service.

4. UCSDMCS by SLLAB

The club website serves three audiences from one Next.js deployment: the public, players using Fixture on the web, and officers maintaining the season's data (Figure 3).



A. Schedule and scores, desktop



B. Standings, mobile

FIGURE 3. UCSDMCS by SLLAB

The site is live at `https://www.ucsdmcs.com`.

4.1. The public site

The schedule lists each game with the opponent's crest, home or away, the field and the result, beneath a season summary of record, win percentage, streak, home and away

records and goal difference. Standings come from the league's SportsEngine service. The site also carries the roster, tryout information with a registration QR code, and the club's description. The schedule is published as an iCalendar feed (Desruisseaux 2009), so a game added in the dashboard appears in subscribers' calendars without anyone re-sharing it.

Caching follows how the data changes. Public pages are cached without expiry and invalidated by a single tag whenever the dashboard writes, because club data stays correct until someone changes it. Standings are the exception: league administrators enter results on their own schedule, so standings are refreshed at most once a day.

4.2. The team app and the dashboard

The web team app is a port of Fixture served at `app.ucsdmcs.com`. A single deployment serves both the club site and the app: a proxy step reads the request's host and serves the team section from the root, so each has its own address without a separate deployment.

The web and iOS clients share one session format, a payload signed with HMAC-SHA256 (Krawczyk, Bellare, and Canetti 1997). The website stores it in an HTTP-only cookie and the app sends it as a bearer token, so one format and one verification path serve both.

The dashboard lets officers edit teams and crests, the roster, the schedule, practices, socials, tournaments and fields. Every write is a server action that checks the session itself rather than relying on the page layout, because server actions can be invoked directly.

5. Data Model

The schema separates durable identity from seasonal participation, and generic events from their type-specific details. Figure 4 shows the core tables.

People are not roster entries. A person exists once. A roster entry is that person on one team in one season, with that season's number, position and class. A player's history, and later their career statistics and alumni status, follow from this separation.

Events are a supertype. Every event has a team, a season, a type and a time, so the schedule, availability and reminders work identically for all of them. Details specific to games or practices live in subtype tables keyed by the event.

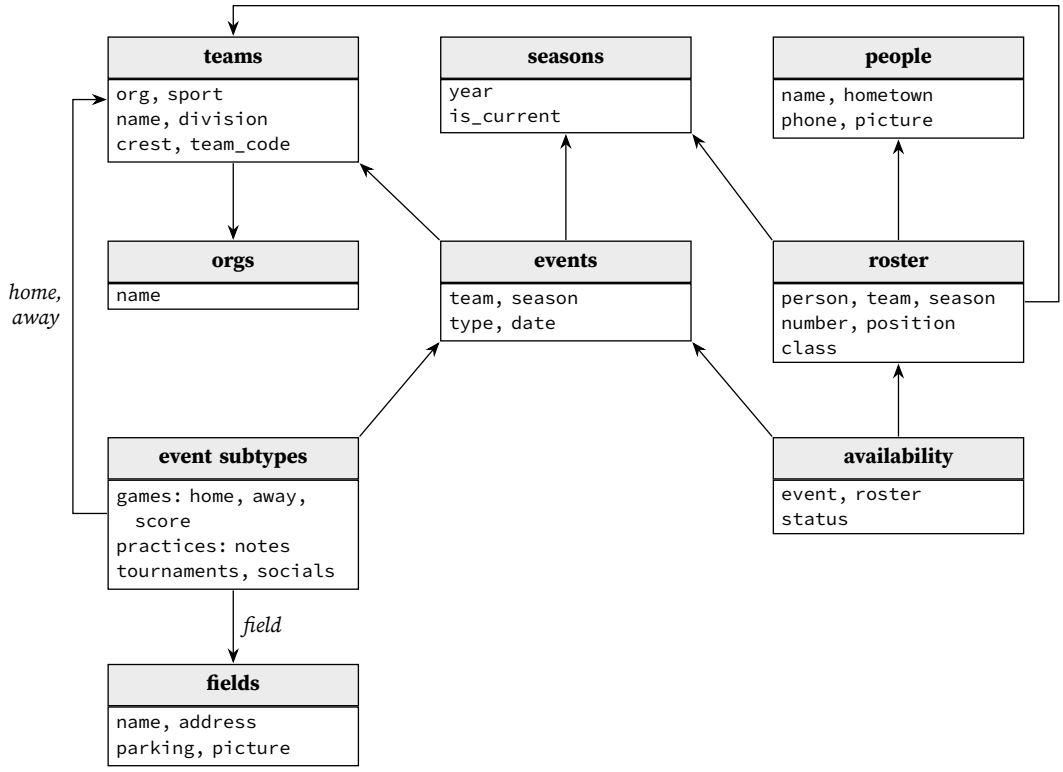


FIGURE 4. Core tables of the schema

Arrows point from a row to the row it references. Games, practices, tournaments and socials each have their own table keyed by the event; they are grouped here.

Availability belongs to a roster entry and an event. A uniqueness constraint on the pair makes each response an upsert, so responses cannot be duplicated.

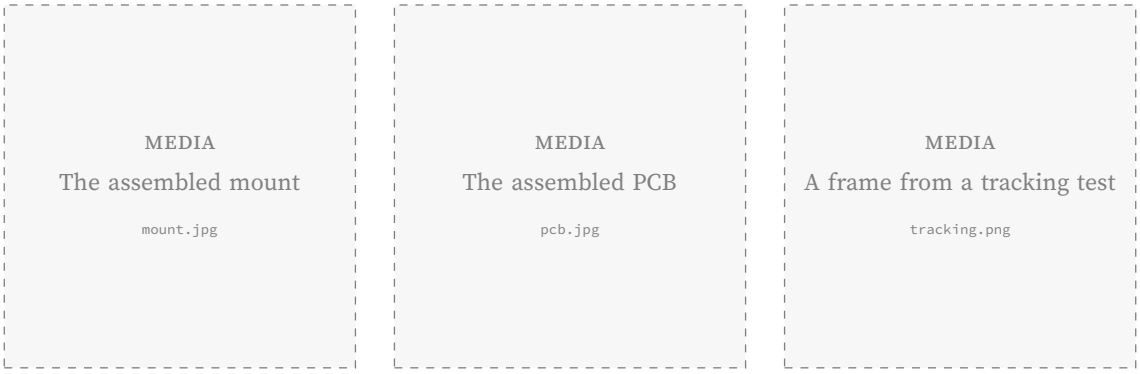
Games name two teams. A game records a home and an away team rather than "us" and an opponent. It reads as home or away depending on whose side it is viewed from, so the same row can serve both clubs once both are on the platform.

Constraints live in the database. Allowed positions, classes and statuses are check constraints, and a partial unique index allows only one current season.

Limitations. Several of these choices were right for one team and must change for many. The current season is global rather than per team, positions are those of soccer, and joining relies on shared team and captain codes rather than personal accounts. Section 8 describes how each changes.

6. Film by SLLAB and the Tracking Mount

Film turns a phone into an unattended camera operator. The phone records on its ultra-wide camera while a detection model locates play in the frame, and a motorized base turns to keep it centered (Figure 5).



A. Mount with phone B. Controller board C. Tracking test

FIGURE 5. Hardware by SLLAB

6.1. The control loop

Figure 6 shows the path from camera frame to motor.

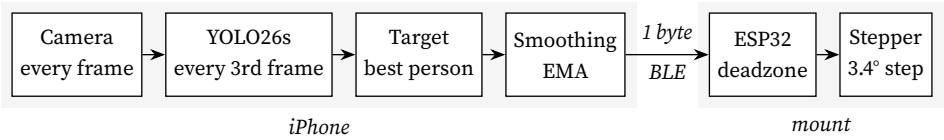


FIGURE 6. The tracking control loop

Detection. Frames are passed to a YOLO26s model (Ultralytics 2026) through Apple's Vision and Core ML frameworks. Only every third frame is processed, which keeps inference within budget while the phone also records. The target is the most confident person detection above a confidence of 0.4.

Error signal. Let $x_c \in [0, 1]$ be the target's horizontal center in normalized image coordinates. The offset from the center of the frame is

$$o_t = 2\,(x_c - 0.5) \in [-1, 1], \tag{2}$$

and it is smoothed with an exponential moving average, so that a single noisy detection does not swing the camera:

$$s_t = 0.7\, s_{t-1} + 0.3\, o_t. \tag{3}$$

When nothing is detected, the average resets and the base holds still until play is found again.

Protocol. The smoothed offset is quantized to one signed byte, $b = \operatorname{trunc}(127\, s_t)$, and written over Bluetooth Low Energy without acknowledgement. A single byte with no round trip keeps latency low, and a lost command is superseded by the next one a fraction of a second later.

Firmware. The ESP32 keeps only the newest command. Commands that arrive while the motor is moving overwrite one another instead of queueing, so the mount never moves toward where the target *was*. Offsets with $|b| \le 15$, about 12% of full scale, are ignored, so the camera holds steady when play is roughly centered instead of hunting back and forth. Outside this deadzone, the motor moves 15 microsteps toward the target. With the driver at $1/8$ microstepping on a 200-step motor, each move turns the camera by

$$\frac{15}{200 \times 8} \times 360^\circ \approx 3.4^\circ. \tag{4}$$

6.2. Hardware

The base is a NEMA 17 stepper motor driven by a TMC2209, whose quiet mode keeps motor noise off the recording's audio. The controller is a custom two-layer, through-hole board of about 45×65 mm, designed in EasyEDA, with the ESP32 and the driver in header sockets so either can be replaced without soldering. A 12 V supply feeds both the motor and the ESP32, with a 100 µF bulk capacitor to absorb current spikes when the motor steps. The housing, shaft coupler and phone holder are 3D-printed. The design progressed from a hand-wired breadboard to a printed base and then to the custom board, and the enclosure and board were refined with a mechanical engineer and a systems engineer. The complete system costs about $400.

6.3. Recording, library and upload

Film records immediately or on a timer. Videos and thumbnails are kept in the app's own library rather than the camera roll and are excluded from iCloud backup, with metadata stored in SwiftData. Videos are uploaded to YouTube as unlisted through the YouTube Data API's resumable upload protocol (Google 2026), and the resulting link is saved with the video.

6.4. The detection model

The shipping model is YOLO26s, tracking people. To track the ball instead, the model is being fine-tuned with Ultralytics on a public soccer-ball dataset (Roboflow Universe 2022), at 640×640 for 100 epochs on the Apple GPU. A second dataset, which labels the ball, goalkeepers, players and referees (Roboflow Universe 2026), is prepared for later work. The trained model is exported to Core ML with non-maximum suppression built in, so the app receives final detections.

6.5. Before release

Film is a working prototype, and four changes precede its release. Upload authorization moves to the platform's API, so that each team uploads to its own channel and the app holds no credentials. Uploads stream from disk instead of loading whole files into memory, which full-length matches require. Tracking moves from the most confident player to the ball. Finally, each video is linked to its game through the shared backend, so that film appears in Fixture.

7. Engineering Practice

One repository. The project began as five repositories, which were merged into one with their full history. A schema change and every client it affects can now land in a single commit. Each application keeps its own folder and toolchain, and code moves into shared packages only when a second consumer needs it.

The schema is code. The live database was captured as a baseline migration. Every later change is an ordered SQL file applied with the Supabase command-line tool, never an edit in the dashboard. The tool's version is pinned, because a major release changed the default target of database commands from the hosted project to the local one.

Secrets stay out of version control. Every repository's history was scanned for keys before the merge, and only templates of required variables are committed. The module holding the server's secret database key is marked server-only, so importing it from client code is a build error rather than a convention.

Large artifacts stay out of version control. Training data, runs and weights are excluded. The repository holds the scripts and dataset definitions needed to re-create them.

Continuous integration. A path-filtered workflow builds and lints each web application when it changes.

8. Roadmap

The next stage turns one team's tools into a platform that any club can join without a developer. Figure 7 shows the target structure, and the steps below are in order.

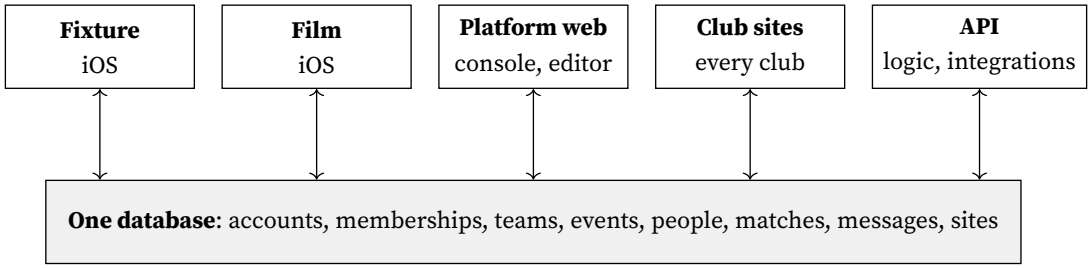


FIGURE 7. Target structure of the platform

Every client uses one backend, and platform logic lives in one API.

1. *Accounts and access.* Verified sign-in by phone or email, and a membership table relating each account to its teams and roles. Access rules in the database then scope every read and write to the caller's teams, and captain becomes a role granted by another captain rather than a shared code. Fixture's mandatory-update mechanism moves every player to the new version before the old paths are removed.
2. *A platform API.* Server operations move from the club's website into a dedicated service with versioned routes, so no application depends on a club's domain. Messaging becomes a subsystem with templates, audience segments, scheduled sends, consent and delivery records.
3. *Multi-team data.* Seasons per team, sport-specific positions, and a shared directory of schools, teams, crests, fields and leagues that every club references instead of copying. Each new club then improves the data for the others.
4. *Matches and statistics.* Lineups and match events, from which player profiles, career statistics and standings follow.
5. *Club sites as data.* One deployment renders every club's site from its domain, with content stored as typed blocks and themed with the club's colors and crest. The UC San Diego site becomes the first set of rows instead of a codebase.
6. *Graphics, gear and dues.* Match-day and result graphics generated from match data; kit orders that collect sizes against roster numbers; and dues collected through each club's own payment account.
7. *Film on the platform.* Videos attached to their events, from Film or any other source, and visible in Fixture.

References

Desruisseaux, Bernard. 2009. "Internet Calendaring and Scheduling Core Object Specification (iCalendar)." RFC 5545, Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc5545.

Google. 2026. "YouTube Data API: Resumable Uploads." Accessed November 2026. https://developers.google.com/youtube/v3/guides/using_resumable_upload_protocol.

Krawczyk, Hugo, Mihir Bellare, and Ran Canetti. 1997. "HMAC: Keyed-Hashing for Message Authentication." RFC 2104, Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc2104.

Roboflow Universe. 2022. "Soccer Ball Dataset, Version 7." CC BY 4.0. https://universe.roboflow.com/t-kefeng-gmail-com/soccer-ball-kjoyy/dataset/7.

Roboflow Universe. 2026. "Football Players Detection Dataset, Version 20." CC BY 4.0; accessed January 2026. https://universe.roboflow.com/roboflow-jvuqo/football-players-detection-3zvbc/dataset/20.

Supabase. 2026. "Supabase Documentation: Database Migrations and Row Level Security." Accessed November 2026. https://supabase.com/docs.

Twilio. 2026. "Messaging Services." Accessed November 2026. https://www.twilio.com/docs/messaging/services.

Ultralytics. 2026. "Ultralytics YOLO Documentation." Accessed November 2026. https://docs.ultralytics.com.

Appendix A. Development Timeline

Table A1 summarizes development, from the projects' commit histories.

TABLE A1. Development timeline, 2026

Month	Milestone
April	Film begins: camera capture and live preview.
June	Detection runs on device, from frame capture to bounding boxes.
July	The phone steers the mount over Bluetooth.
August	Recording moves to the ultra-wide camera, and tracking works at 10–15 ft. In-app library and YouTube upload. The sllab.io site launches.
September	The club website launches with the dashboard, live standings and tryouts. Fixture enters beta at version 1.0.0, followed by its web version.
October	Text-message reminders and Keychain-backed captain sessions. Attendance tracking for captains. A revised schema, unit tests and mandatory updates.
November	Past events in Fixture. The mount's firmware, board and CAD files are published. The five repositories are merged into one.